



ESTRUTURA DE DADOS

ESTRUTURA DE DADOS - AULA INICIAL

Prof. Me. Gustavo Ferreira Palma

QUEM SOU EU?

FORMAÇÃO ACADÊMICA

- Técnico em Informática - Etec Waldyr Duron Júnior
- Tecnólogo em Análise e Desenvolvimento de Sistemas - FATEC Ourinhos
- Especialista Em Eletrônica Embarcada - UNISAL
- Mestre em Inteligência Computacional Aplicada - UTFPR

EXPERIÊNCIA PROFISSIONAL

- Analista de Tecnologia - Chiptronic Eletrônica do Brasil (2011-2019)
- Professor de Graduação - Eduvale Avaré (2017-2019)
- Analista de Desenvolvimento - Stoneridge Brasil (2019-2020)
- Consultor Tecnológico - Instituto de Pesquisas Eldorado (2020)
- Professor de Graduação e Pós-Graduação - UNISAL (2023)

QUEM SÃO VOCÊS?

O QUE VAMOS ESTUDAR?

- Estrutura de Dados utilizando linguagem C
 - Tipos Primitivos de dados
 - Gerenciamento de memória
 - Vetores e Matrizes
 - Ponteiros
 - Tipos Abstratos de dados
 - Funções
 - Arquivos
 - Algoritmos de ordenação
 - Algoritmos de busca
 - Filas
 - Pilhas
 - Listas
 - Entre outros...

MÉTODOS DE AVALIAÇÃO

- Majoritariamente avaliações práticas;
- Se/Quando Necessário, Avaliação escrita;

COMBINADOS DE OURO

- Nosso Curso é PRESENCIAL, então para ter presença é preciso estar PRESENTE;
- Abono de Faltas, somente com atestado e casos previstos;
- Nossa Sala de Aula é um ambiente seguro, fiquem a vontade para perguntar, tirar dúvidas, fazer experimentos, é um prazer ajudar e contribuir para o crescimento de cada um de vocês;
- Evitem Conversas Paralelas, por mais que pareça que temos muito tempo, nosso tempo em sala de aula é curto, aproveitem;

COMBINADOS DE OURO

- Se tivermos alguma atividade manuscrita, caprichem na caligrafia, não sou farmacêutico, se eu não conseguir ler, não tem nota;
- Lembrem-se em toda e qualquer atividade que desenvolvermos, eu sei que o Chat-GPT sabe como resolver, nesse momento quero saber de vocês, não tenham medo de errar, de testar, de experimentar, perguntem se necessário, desenvolvam suas capacidades.
- Guardem as conversas para momentos extra-classe, todos aqui são amigos, mas durante as aulas, o importante é ouvir, perguntar e interagir, com o professor.

DÚVIDAS?

PORQUE LINGUAGEM C?

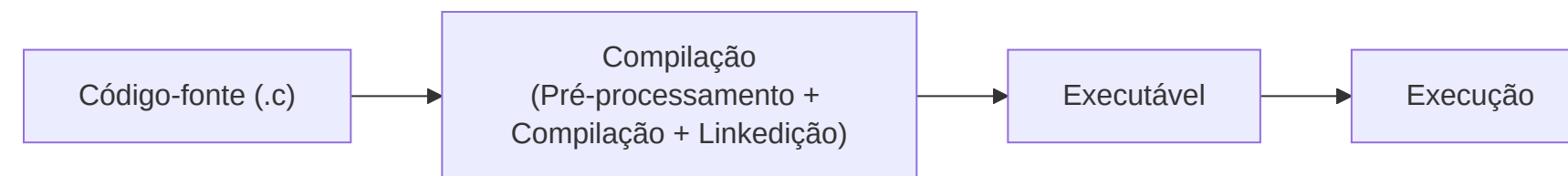
PORQUE LINGUAGEM C?

- Muitos podem pensar que a Linguagem de programação C, é uma Tecnologia obsoleta, muito difícil e extremamente ultrapassada, contudo isso não é verdade
- A Linguagem de programação C, é considerada uma linguagem clássica, afinal faz 50 anos que ela foi criada. Contudo, isso não significa que ela não é atualizada
- A última versão da linguagem C, a C23 foi lançada em 2024, o que faz dela uma linguagem atual e amplamente utilizada em diferentes ambientes indo desde a criação de sistemas operacionais até o desenvolvimento de sistemas embarcados e aplicações de alto desempenho

PORQUE LINGUAGEM C?

- A linguagem C foi pensada para ter um nível baixo de abstração, mantendo-a próximo ao hardware;
- Essas duas características principais, a tornam a melhor escolha para o ensino de técnicas de programação e demonstração de conceitos
- Durante as aulas vocês podem usar qualquer ambiente de desenvolvimento e qualquer compilador C, nós vamos nos ater as bases e a compreensão dos principais recursos

PROCESSO DE COMPILAÇÃO



TIPOS PRIMITIVOS DE DADOS

- Os tipos primitivos são os tipos de dados fundamentais fornecidos pela linguagem C. Eles representam valores básicos, como números inteiros, números de ponto flutuante e caracteres, servindo como base para a construção de estruturas de dados mais complexas.
- Cada tipo possui características próprias, como tamanho em memória, faixa de valores e precisão, fatores que influenciam diretamente o desempenho e o consumo de memória de um programa. A escolha do tipo adequado é uma decisão importante durante o desenvolvimento, pois permite representar os dados de forma eficiente e segura.
- Na linguagem C, os tamanhos dos tipos podem variar de acordo com a arquitetura do computador e o compilador utilizado. Entretanto, atualmente, em sistemas de 64 bits, existe um conjunto de tamanhos que se tornou amplamente adotado e que será utilizado como referência ao longo desta disciplina.

TIPOS PRIMITIVOS DE DADOS

Tipo	Tamanho típico	Faixa de valores	Observações
char	1 byte	-128 a 127	0 a 255 para 'unsigned char'
short	2 bytes	-32.768 a 32.767	0 a 65.535 para unsigned short
int	4 bytes	-2.147.483.648 a 2.147.483.647	0 a 4.294.967.295 para unsigned int
long	8 bytes	$\pm 9,22 \times 10^{18}$	0 a 18.446.744.073.709.551.615 para unsigned long
long long	8 bytes	$\pm 9,22 \times 10^{18}$	0 a 18.446.744.073.709.551.615 para unsigned long long
float	4 bytes	$\pm 3,4 \times 10^{38}$	menor valor positivo: $1,175 \times 10^{-38}$ maior valor finito: $3,403 \times 10^{38}$ precisão aproximada: 6–7 dígitos
double	8 bytes	$\pm 1,7 \times 10^{308}$	menor valor positivo: $2,225 \times 10^{-308}$ maior valor finito: $1,798 \times 10^{308}$ precisão aproximada: 15–16 dígitos
long double	16 bytes	até $\pm 1,1 \times 10^{4932}$	menor valor positivo: $\approx 3,362 \times 10^{-4932}$ maior valor finito: $\approx 1,190 \times 10^{4932}$ precisão aproximada: 18–21 dígitos
_Bool (bool)	1 byte	`false` ou `true`	---

TIPOS PRIMITIVOS DE DADOS

- Cálculo para Valores Sinalizados

$$\text{Faixa}_{\text{signed}} = -2^{N-1} \text{ até } 2^{N-1} - 1$$

- Cálculo para Valores não sinalizados

$$\text{Faixa}_{\text{unsigned}} = 0 \text{ até } 2^N - 1$$

TIPOS PRIMITIVOS DE DADOS

- Cálculo para Valores Sinalizados - Exemplo 8 bits

signed: -2^7 até $2^7 - 1$
 -128 até 127

- Cálculo para Valores não sinalizados - Exemplo 8 bits

unsigned: 0 até $2^8 - 1$
 0 até 255

TIPOS PRIMITIVOS DE DADOS E O C99

- Embora a linguagem C possua tipos como char, short, int e long, o tamanho desses tipos pode variar de acordo com a arquitetura e o compilador utilizado. Para resolver esse problema, o padrão C99 introduziu o cabeçalho `stdint.h`, que define tipos inteiros com tamanhos fixos e portáteis.
- Esses tipos são criados utilizando a palavra-chave `typedef`, que permite definir um novo nome (apelido) para um tipo já existente. Dessa forma, um programador pode utilizar tipos como `uint8_t` ou `int32_t` sabendo exatamente quantos bits eles ocupam, independentemente da plataforma em que o código será compilado.
- Essa padronização tornou o desenvolvimento de bibliotecas, sistemas operacionais, sistemas embarcados e protocolos de comunicação muito mais seguro e portátil.

TIPOS PRIMITIVOS DE DADOS E O C99

Tipo	Tamanho	Faixa de Valores
int8_t	8 bits	-128 a 127
uint8_t	8 bits	0 a 255
int16_t	16 bits	-32.768 a 32.767
uint16_t	16 bits	0 a 65.535
int32_t	32 bits	-2.147.483.648 a 2.147.483.647
uint32_t	32 bits	0 a 4.294.967.295
int64_t	64 bits	-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807
uint64_t	64 bits	0 a 18.446.744.073.709.551.615

ENTÃO, COMO CRIAR NOVOS TIPOS DE DADOS, PORTÁVEIS OU MAIS AMIGÁVEIS?

ENTÃO, COMO CRIAR NOVOS TIPOS DE DADOS, PORTÁVEIS OU MAIS AMIGÁVEIS?

- A resposta é simples, não criamos. Nós apenas criamos apelidos (alias) para tipos existentes, o que facilita o entendimento de outros desenvolvedores.
- A avaliação dos apelidos (alias), é feita durante a etapa de pré processamento:

```
1
2 //...
3 typedef unsigned char uint8_t;
4 typedef signed int int32_t;
5 //...
```

ENTÃO, COMO CRIAR NOVOS TIPOS DE DADOS, PORTÁVEIS OU MAIS AMIGÁVEIS?

- A resposta é simples, não criamos. Nós apenas criamos apelidos (alias) para tipos existentes, o que facilita o entendimento de outros desenvolvedores.
- A avaliação dos apelidos (alias), é feita durante a etapa de Compilação do código:

```
1
2 // ...
3 typedef unsigned char uint8_t;
4 typedef signed int int32_t;
5 // ...
```

ENTÃO, COMO CRIAR NOVOS TIPOS DE DADOS, PORTÁVEIS OU MAIS AMIGÁVEIS?

- Ainda tratando de dados que tornam o código mais amigável, podemos citar as macros
- As macros, são meios de de melhorar a legibilidade, e em alguns casos podemos melhorar até o desempenho de um software;
- As macros nos ajudam a evitar uma prática muito comum no desenvolvimento de softwares, os chamados 'números mágicos' ou mesmo 'valores mágicos'.

```
1
2      //...
3      socket = conectar("192.168.1.10", 8080);
4      //...
```

```
1
2      #define ENDERECO_IP "192.168.1.10"
3      #define PORTA 8080
4      //...
5      socket = conectar(ENDERECO_IP,PORTA);
6      //...
```

OBRIGADO!

Podem me encontrar aqui:

- gustavo.palma@unisal.edu.br
- www.gustavopalma.com.br